



Universidad Nacional de Ingeniería
Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación
Programación Orientada a Objetos

Práctica calificada 2

CC211

22/09/2025 Tiempo: 2 horas

Ciclo: 2025-2

Normas:

1. El alumno entregará esta hoja de examen debidamente llenada con sus datos.
2. La solución de la prueba se guardarán en **Escritorio**, carpeta: **ApellidoNombreCodigo** (sin espacios en blanco), la pregunta **n** se guardará en el archivo: **n.c** ($n = 1, 2, ..$).
3. No se permite: El uso de celulares, internet, USB, ingresar después de 15 min. de iniciado el examen ni salir antes de la hora de ización.
4. Todo acto anti-ético será amonestado y registrado en el historial del estudiante.

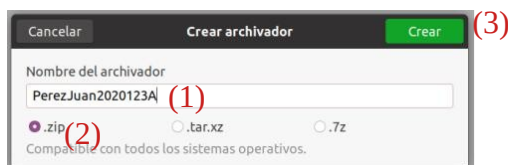
Directrices

- 0) Recibirá un mensaje con un **link** a una carpeta que contiene las preguntas
- 1) cree su directorio de trabajo **dt** y ubicarse en él.
- 2) Baje las preguntas del **link** a **dt**
- 3) Programe los archivos Test.java y Chancha.java
- 4) Comprimir los archivos **.java** en el archivo **ApellidoNombre.zip**
- 5) Subir **ApellidoNombre.zip** al **link** anterior.

Ayuda para comprimir archivos .java

Comprimir en Ubuntu

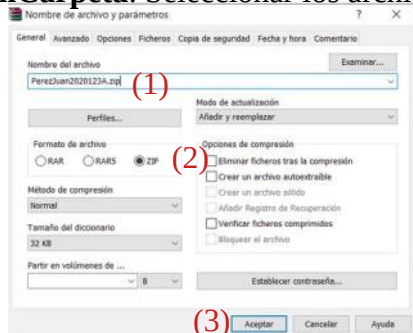
En **miCarpeta**: Seleccionar los archivos a comprimir → Click derecho → **Comprimir...**:



En **miCarpeta** aparece: **PerezJuan2020123A.zip**

Comprimir en Windows

En **miCarpeta**: Seleccionar los archivos → Click derecho → **Añadir al Archivo**:



// En **miCarpeta** aparece: **PerezJuan2020123A**

Subir ApellidoNombre.zip al link suministrado.

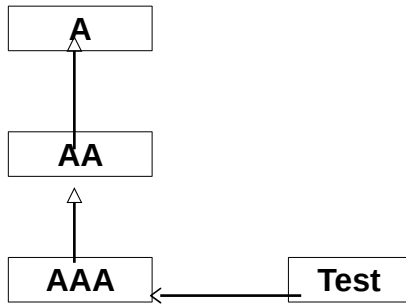
Requerimientos de programación

Los programas deben cumplir:

- 1) Satisfacer el diseño POO solicitado y el formato de presentación en el monitor.
- 2) Optimidad y orden

El incumplimiento de estos requerimientos penaliza la nota,

1) [5 pts] Dado el diagrama de clases:



En el archivo **Test.java** programe las clases del diagrama para confirmar (en el monitor) las reglas:

- 1) Al crear un objeto de una subclase, se ejecutan todos los constructores de la jerarquía de sus super clases.
- 2) Desde un constructor se puede llamar a otros métodos de la clase.

2) [15 pts] Vamos a organizar una fiesta de fin de ciclo, para ello haremos una chancha, que será controlada por la clase **Chancha** con la siguiente salida en el monitor:

Chancha para la fiesta de fin de ciclo. Monto >= 2000

Estudiantes				Total	Mayores aportadores (de dinero)
Cuota	E0	E1	E2		
1	68	62	60	190	E0
2	63	64	67	194	E2
3	70	63	63	196	E0
4	64	69	70	203	E2
(1) 5	61	61	61	(2) 183	E0 E1 E2 (3)
6	69	60	68	197	E0
7	69	65	66	200	E0
8	67	60	64	191	E0
9	69	66	61	196	E0
10	68	62	62	192	E0
11	64	67	63	194	E1
(4) Total 732 699 705				(5) 21360	E0 (6)

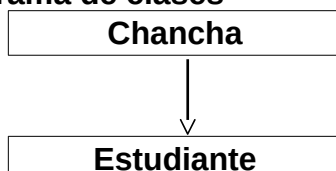
Descripción de las 6 áreas:

Nombres de variables en rojo

Cuota	E0	E1	E2	Total	Mayores aportadores	
(1) 1	68	62	60	(2) 90	E0	(3)
nFila	aporte			aporteFila	aportes[..]	
aporte = 60 + ran.nextInt(11);						
(4) Total 732 699 705				(5) 21360	E0	(6)
estudiantes(i).totalAporte				Estudiante.	aportes[..]	
				totalAportes		

Las área y flechas serán descritas en la clase Chancha

Diagrama de clases



Chancha

```
+ main(){
    Random ran = new Random();
    int monto = 2000;           // monto de la chancha
    int nEst = 2+ran.nextInt(5); // número de estudiantes
}

// Crear estudiantes y reportar el título

// Aportes
int aporte, aporteFila, aporteMax, aporteTotal=0;
int aportes[ ] = new int[nEst];
int nFila = 1;
do{
    System.out.printf(" %2d ", nFila++);
    aporteFila = 0;
    aporteMax = 0;
(1)  for(i=0; i<nEst; i++){
        aporte = 60 + ran.nextInt(11);
        // imprimir aporte;
        // calcular aporteFila; (1) → (2)
        // calcular aporteMax; (1) → (3)
        // aportes[i] = aporte; (1) → (3)
        // ejecutar: estudiantes[i].aportar(aporte); (1) → (4)
    }
(2)  // imprimir aporteFila;
    // calcular aporteTotal; (para finalizar do-while)
(3)  for(i=0; i<nEst; i++)
        // imprimir estudiantes[i].nombre de mayores aportadores: aportes[i] == aporteMax;
    } while (aporteTotal<monto);

// Total Aportes
System.out.print("Total");
aporteMax = 0;
(4)  for(i=0; i<nEst; i++) {
        // ejecutar: aportes[i] = estudiantes[i].aportar(aporte); (4) → (6)
        // imprimir aportes[i]
        // calcular aporteMax; (4) → (6)
    }
(5)  // imprimir Estudiante.totalAportes;
(6)  for(i=0; i<nEst; i++)
        // imprimir estudiantes[i].nombre de mayores aportadores: aportes[i] == aporteMax;
    System.out.println();
}
```

Estudiante

```
static int estudiantes, totalAportes;
int codigo;
String nombre;
int totalAporte;
Estudiante(int codigo, String nombre){
    this.codigo = codigo;
    this.nombre = nombre;
    estudiantes++;
}
void aportar(int aporte){
    totalAporte += aporte;
    totalAportes += aporte;
}
```